Research Report 06:

Whisper ASR Integration
Automatic Transcription for Low-Resource Sinhala Language

Voice-of-Hands Research Team

April 15, 2026

Abstract

This report documents the integration of OpenAI's Whisper automatic speech recognition (ASR) system for Sinhala language transcription in the Voice-of-Hands dataset creation pipeline. We evaluate Whisper's performance on Sri Lankan parliamentary speech, compare model sizes, analyze word-level timestamp accuracy, and address challenges specific to low-resource language ASR. Our results demonstrate 95%+ transcription success rate with the medium model, validating Whisper as a viable solution for Sinhala speech-to-text in broadcast domains.

Contents

1 Introduction

1.1 Automatic Speech Recognition for Low-Resource Languages

Automatic speech recognition for low-resource languages presents unique challenges:

- **Limited training data**: Few large-scale annotated speech corpora
- **Script complexity**: Non-Latin scripts (Sinhala: U+0D80–U+0DFF)
- **Morphological richness**: Agglutinative word formation
- **Domain mismatch**: Generic models trained on Western languages
- **Pronunciation variability**: Regional accents, code-switching

1.2 Why Whisper?

OpenAI's Whisper [**?**] offers several advantages for low-resource languages:

1. **Multilingual pre-training**: Trained on 680,000 hours covering 99 languages including Sinhala
2. **Robust to noise**: Encoder-decoder Transformer architecture with attention handles broadcast-quality audio
3. **Zero-shot capability**: No fine-tuning required for Sinhala transcription
4. **Word-level timestamps**: Enables precise audio-text alignment
5. **Open-source**: Freely available (MIT license), reproducible
6. **Multi-task training**: Joint optimization for speech recognition, translation, and language identification

1.3 Objectives

1. Evaluate Whisper model sizes (tiny, base, small, medium, large) on Sinhala
2. Measure transcription accuracy on parliamentary broadcast speech
3. Analyze word-level timestamp precision
4. Assess computational requirements (GPU memory, inference time)
5. Identify failure modes and error patterns
6. Validate integration with multimodal dataset pipeline

2 Whisper Architecture Overview

2.1 Model Architecture

Whisper employs an encoder-decoder Transformer architecture:
Encoder:

- Input: Log-mel spectrogram (80 channels)
- Conv1D layers with GELU activation

- Sinusoidal positional encoding
- Multi-head self-attention layers
- Output: Audio feature embeddings

Decoder:

- Input: Text tokens (BPE with 50,000 vocabulary)
- Causal multi-head self-attention
- Cross-attention to encoder outputs
- Output: Token probabilities

2.2 Model Variants

Table 1: Whisper Model Comparison

Model	Parameters	Layers	d_model	VRAM
Tiny	39M	4	384	∼1 GB
Base	74M	6	512	∼1 GB
Small	244M	12	768	∼2 GB
Medium	**769M**	**24**	**1024**	**∼5 GB**
Large	1550M	32	1280	∼10 GB

2.3 Preprocessing Pipeline

1. **Audio loading**: Librosa reads WAV file
2. **Resampling**: Resample to 16 kHz (if needed)
3. **Normalization**: Scale to [-1, 1] range
4. **Padding/Trimming**: Chunk to 30-second segments
5. **Mel spectrogram**: 80-channel log-mel filterbank
6. **Inference**: Encoder-decoder forward pass
7. **Decoding**: Beam search or greedy decoding

3 Experimental Setup

3.1 Hardware Configuration

- **CPU**: Intel Core i7 (for small-model inference)
- **GPU**: NVIDIA GPU with CUDA support (for medium/large models)
- **RAM**: 16 GB system memory
- **Storage**: SSD for audio clip caching

3.2 Software Stack

```
# Installation
pip install openai-whisper

# Dependencies
# - torch (PyTorch)
# - numpy
# - scipy
# - ffmpeg-python
# - librosa
```

3.3 Test Dataset

- **Source**: 732 audio clips from 61-minute parliamentary session
- **Format**: 16 kHz mono PCM WAV
- **Duration**: 5.0 seconds per clip
- **Language**: Sinhala (primary), some Tamil and English code-switching
- **Speakers**: Multiple parliamentary speakers (male/female)
- **Domain**: Political discourse, legislative proceedings

4 Model Evaluation

4.1 Transcription Quality

Methodology: Manual review of 100 random transcriptions

Table 2: Transcription Quality by Model Size

Model	Perfect	Minor Errors	Major Errors	Failed	Success Rate
Tiny	52%	28%	15%	5%	80%
Base	64%	22%	11%	3%	86%
Small	74%	18%	6%	2%	92%
Medium	**82%**	**13%**	**3%**	**2%**	**95%**
Large	85%	12%	2%	1%	97%

Error categories:

- **Perfect**: 100% accurate transcription
- **Minor errors**: 1–2 word errors (homophones, particles)
- **Major errors**: >2 word errors or semantic changes
- **Failed**: No output or non-Sinhala output

Selected Model: **Medium** (769M parameters)
Rationale:

- 95% success rate (acceptable for dataset creation)
- 5 GB VRAM (available on consumer GPUs)

- 82% perfect transcriptions
- Marginal improvement from large model (97% vs. 95%) does not justify 2× memory and 1.5× inference time

4.2 Inference Performance

Table 3: Inference Performance (5-second audio clips)

Model	CPU Time	GPU Time	VRAM	Speed Factor
Tiny	2.5s	0.4s	0.8 GB	12.5× (GPU)
Base	4.1s	0.6s	1.1 GB	8.3×
Small	8.3s	1.2s	2.3 GB	4.2×
Medium	**18.5s**	**2.4s**	**4.8 GB**	**2.1×**
Large	35.2s	4.1s	9.6 GB	1.2×

Speed factor: Real-time multiple (e.g., 2.1× means processing 5s of audio takes 2.4s).
Batch processing: 732 clips with medium model

$$\text{Total time (GPU)} = 732 \times 2.4s = 1757s \approx 29 \text{ minutes} \quad (1)$$

$$\text{Total time (CPU)} = 732 \times 18.5s = 13{,}542s \approx 3.8 \text{ hours} \quad (2)$$

4.3 Word-Level Timestamp Accuracy

Whisper provides word-level timestamps through forced alignment:

```
import whisper

model = whisper.load_model("medium")

result = model.transcribe(
    "audio_clip.wav",
    language="si",
    word_timestamps=True
)

# result['segments'] contains word-level timing
for segment in result['segments']:
    for word_info in segment['words']:
        word = word_info['word']
        start = word_info['start']
        end = word_info['end']
        confidence = word_info.get('probability', 1.0)
        print(f"{word}: [{start:.2f}s - {end:.2f}s]")
```

Timestamp precision validation:
Conclusion: Timestamp accuracy sufficient for coarse alignment (sub-second precision).

5 Sinhala-Specific Challenges

5.1 Challenge 1: Script Complexity

Sinhala abugida script presents tokenization challenges:
Example word decomposition:

Table 4: Word Timestamp Accuracy (N=50 clips, 250 words)

Metric	Value
Mean timestamp error	0.12s
Median timestamp error	0.08s
Std deviation	0.15s
90th percentile error	0.28s
Words with <0.1s error	68%
Words with <0.2s error	88%

- Word: (parliament)
- Unicode: U+0DB4 U+0DCF U+0DBB U+0DCA U+200D U+0DBD U+0DD2 U+0DB8 U+0DD9 U+0DB1 U+0DCA U+0DAD U+0DD4 U+0DC0
- 14 code points for single word

Whisper handling:

- Byte-pair encoding (BPE) vocabulary includes Sinhala subword units
- Training on 680K hours ensures sufficient Sinhala exposure
- Output: Proper Unicode Sinhala text

5.2 Challenge 2: Agglutinative Morphology

Sinhala words can agglutinate multiple morphemes:

- = + + (say + verb noun + that)
- = + (parliament + locative)

Impact on ASR:

- Long words (15–25 characters common)
- Ambiguous word boundaries
- High out-of-vocabulary rate for word-based models

Whisper advantage: Subword tokenization handles morphological complexity naturally.

5.3 Challenge 3: Code-Switching

Parliamentary speech frequently switches between Sinhala, Tamil, and English:

Example utterance:

" budget speech agree but implementation question "
(I agree with the budget speech but there are questions about implementation)

Whisper behavior:

- Detects dominant language (Sinhala)
- Transcribes English words in Latin script

- Occasional confusion on short English fragments

Solutions attempted:
1. Language forcing:

```
result = model.transcribe(audio, language="si")
# Forces Sinhala, may miss English words
```

2. Multilingual mode:

```
result = model.transcribe(audio, language=None)
# Auto-detects language per segment
```

Selected: Language forcing (`language="si"`) for consistency, accepting that English words may be transliterated into Sinhala script.

5.4 Challenge 4: Domain-Specific Vocabulary

Parliamentary proceedings contain specialized terminology:

- Legal terms: (judiciary), (constitution)
- Procedural: (speaker), (presentation)
- Acronyms: ... (SDG), .. (UNP - political party)

Whisper performance on domain terms:

Table 5: Domain Terminology Accuracy

Term Category	Correct	Accuracy
Common legal terms	42/50	84%
Procedural terms	38/50	76%
Acronyms	18/30	60%
Proper names (politicians)	35/40	88%

Observation: Lower accuracy on acronyms suggests domain fine-tuning would benefit.

6 Integration with Pipeline

6.1 Batch Transcription Script

```
import whisper
import json
from pathlib import Path
from tqdm import tqdm

def transcribe_dataset(audio_dir, output_json, model_size="medium"):
    """
    Transcribe all audio clips in directory.

    Args:
        audio_dir: Directory containing WAV files
        output_json: Output JSON file path
        model_size: Whisper model size
    """
```

```
    model = whisper.load_model(model_size)

    audio_files = sorted(Path(audio_dir).glob("*.wav"))

    results = []

    for audio_path in tqdm(audio_files, desc="Transcribing"):
        try:
            result = model.transcribe(
                str(audio_path),
                language="si",
                word_timestamps=True,
                verbose=False
            )

            transcription = {
                "file": audio_path.name,
                "text": result["text"],
                "language": result["language"],
                "segments": result["segments"],
                "duration": result.get("duration", 0.0)
            }

            results.append(transcription)

        except Exception as e:
            print(f"Error on {audio_path.name}: {e}")
            results.append({
                "file": audio_path.name,
                "text": "",
                "error": str(e)
            })

    # Save results
    with open(output_json, 'w', encoding='utf-8') as f:
        json.dump(results, f, ensure_ascii=False, indent=2)

    print(f"\nTranscribed {len(results)} files")
    print(f"Success: {sum(1 for r in results if 'error' not in r)}")
    print(f"Failures: {sum(1 for r in results if 'error' in r)}")

# Usage
transcribe_dataset(
    audio_dir="multimodal_dataset/audio_clips/",
    output_json="multimodal_dataset/transcriptions/all_transcriptions.
       json",
    model_size="medium"
)
```

6.2 Multimodal Metadata Assembly

```
import json

def create_alignment_metadata(video_clips, audio_clips,
                              transcriptions_json):
    """
    Create aligned multimodal dataset metadata.
```

```
    Output format:
    {
        "clip_001": {
            "video": "path/to/video_001.mp4",
            "audio": "path/to/audio_001.wav",
            "text": "Sinhala transcription",
            "words": [...word-level timestamps...],
            "start_time": 0.0,
            "duration": 5.0,
            "motion_score": 12.5
        },
        ...
    }
    """
    with open(transcriptions_json, 'r', encoding='utf-8') as f:
        transcriptions = json.load(f)

    alignment = {}

    for trans in transcriptions:
        clip_id = Path(trans['file']).stem

        alignment[clip_id] = {
            "video": f"video_clips/{clip_id}.mp4",
            "audio": f"audio_clips/{clip_id}.wav",
            "text": trans['text'],
            "language": trans.get('language', 'si'),
            "words": trans.get('segments', []),
            "duration": trans.get('duration', 5.0)
        }

    with open("alignment_metadata.json", 'w', encoding='utf-8') as f:
        json.dump(alignment, f, ensure_ascii=False, indent=2)

    return alignment

# Create alignment
metadata = create_alignment_metadata(
    video_clips="multimodal_dataset/video_clips/",
    audio_clips="multimodal_dataset/audio_clips/",
    transcriptions_json="multimodal_dataset/transcriptions/
       all_transcriptions.json"
)
```

7 Error Analysis

7.1 Failure Mode Classification

7.2 Common Transcription Errors

1. **Homophone confusion**:
    - (and) ↔ (comrade)
    - (did) ↔ (time)
2. **Particle omission**:

Table 6: Transcription Failure Modes (N=732 clips)

Failure Type	Count	Percentage
Silent audio (no speech)	16	2.2%
Background noise only	8	1.1%
Overlapping speakers	5	0.7%
Non-Sinhala language	3	0.4%
Model error (crash)	2	0.3%
Total failures	**34**	**4.6%**
Successful	**698**	**95.4%**

- Expected: (saying-that)
- Whisper: (saying)

3. **Number transcription**:
   - Spoken: ”2024”
   - Transcribed: (two thousand twenty four)
   - Expected behavior but complicates text matching
4. **Acronym expansion**:
   - Spoken: UNP (UNP acronym)
   - Transcribed: (United National Party)

8 Performance Optimization

8.1 GPU Utilization

```
import torch

# Enable GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"

model = whisper.load_model("medium", device=device)

# Monitor VRAM
if device == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.memory_allocated()/1e9:.2f} GB")
```

VRAM usage (medium model):

- Model weights: 3.1 GB
- Activations (single clip): 1.7 GB
- Total: ∼4.8 GB

Optimization: Process clips sequentially to avoid memory accumulation.

8.2 Batching Strategy

Whisper does not natively support batch inference. For multiple clips:

Option 1: Sequential processing (implemented)

```
for audio_file in audio_files:
    result = model.transcribe(audio_file)
# Simple, memory-safe, slower
```

Option 2: Multi-GPU parallelism (future)

```
from torch.nn import DataParallel

model = DataParallel(model, device_ids=[0, 1])
# Requires code modification, 2   speedup
```

9 Conclusion

9.1 Key Findings

1. **Whisper medium model**: Optimal balance (95% success, 5 GB VRAM, 2.1× real-time)
2. **Sinhala support**: Out-of-the-box performance acceptable for dataset creation
3. **Word timestamps**: Sub-second precision (mean error 0.12s)
4. **Failure rate**: 4.6% (primarily silent/noisy audio, not model errors)
5. **Batch processing**: 732 clips in 29 minutes (GPU) or 3.8 hours (CPU)

9.2 Recommendations

1. **Production deployment**: Use medium model on GPU
2. **Silent clip filtering**: Pre-filter audio with energy detection
3. **Code-switching**: Accept mixed Sinhala/English transcriptions
4. **Post-processing**: Normalize acronyms and numbers
5. **Fine-tuning**: Future work to improve domain terminology

9.3 Future Work

1. **Parliamentary domain fine-tuning**: Improve legal/procedural term accuracy
2. **Speaker diarization**: Segment multi-speaker clips
3. **Tamil support**: Extend to Tamil Sign Language dataset
4. **Error correction**: Post-processing with language model
5. **Confidence calibration**: Filter low-confidence transcriptions